Phase 3 Failure Log

Final result: run 6, on the committed code (`fdb0ce8`), passed all 152 tests and matched all 154 Daily Transaction Files. On the way, the runs found twelve Front End faults (F1 to F12) and three test-design problems, all fixed below.

Only failures seen in a real run belong in the failure log. Predictions and open questions are kept apart.

How to run the tests

The script builds the Front End from `phase2/src`, runs every requirements test in `phase1/tests`, and compares both the terminal output and every Daily Transaction File (DTF) with the expected files.

Windows: from the repository folder, run `phase1\scripts\run_tests.bat` (double-click it or type it in a terminal). It starts `phase1/scripts/run_tests.sh` with Git Bash, so Git for Windows and `g++` must be installed. Without `g++`, first set `CXX` to another C++17 compiler, for example `set "CXX=C:/Program Files/LLVM/bin/clang++.exe"` (run 6 did this).

Git Bash, macOS or Linux: from any folder, run `bash phase1/scripts/run_tests.sh`. Add `--no-build` to test the existing `phase2/frontend.exe` without rebuilding.

For each `*_input.txt` the script:

1. picks the three master files: the test's override in `phase1/test_fixtures/<test id>/` if there is one, otherwise `phase1/shared_test_data/`;
2. runs `phase2/frontend.exe <current_users> <available_games> <game_collection> phase1/results/actual/<test id>` with the input file as standard input;
3. saves the terminal output as `phase1/results/actual/<test id>_actual.txt`; each logout writes `<test id>_session<N>.txt` beside it;
4. compares the terminal output with `<test id>_terminal_expected.txt`, and each session file with `<test id>_dtf_expected.txt` (one session) or `<test id>_session<N>_dtf_expected.txt` (several), ignoring only CR/LF differences;
5. writes one `PASS` or `FAIL <reasons>` line per test and the totals to `phase1/results/comparisons/summary.txt`, and keeps a diff file there for each mismatch.

The results folders are emptied at the start of every run. Exit status: 0 = every test passed, 1 = at least one failed, 2 = could not run.

Run history

Run	Date	Code version	Result	Notes
1	2026-11-01 02:08	`406e6df` plus uncommitted test infrastructure; `Money.cpp`, `buy_13` and SEARCH unchanged	131 total, 0 passed, 131 failed	No `*_terminal_expected.txt` files existed yet, so terminal output was not compared in this run
2	2026-11-06 11:47	`7bce07d` (main after all terminal expected files were committed)	150 total, 49 passed, 101 failed	All 101 failures `TERMINAL_DIFF`, from causes F1 to F11 below; every DTF matched. The 49 passes were not valid: see "Test corrections"
3	2026-11-06 11:52	`7bce07d` code (run 2's build); 50 terminal expected files rewritten from the catalogue	150 total, 0 passed, 150 failed	All failures `TERMINAL_DIFF`; every DTF matched. Same causes as run 2, now also seen in the 49 corrected tests
4	2026-11-06 11:55	`7bce07d` plus the fixes for F1 to F12, not yet committed. Committed unchanged as `fdb0ce8`	150 total, 150 passed, 0 failed	

Run	Date	Code version	Result	Notes
5	2026-11-06 11:56	Same code as run 4; new tests `addcredit_12` and `addcredit_13` (also in `fdb0ce8`)	152 total, 152 passed, 0 failed	The two new tests were also run against a build of `7bce07d` (the code of runs 2 and 3): both failed (F12)
6	2026-11-07 01:56	`fdb0ce8`, committed, clean working tree	152 total, 152 passed, 0 failed	Final confirmation on Windows 11 with clang++ 23.1.3 (`g++` not installed); 154 of 154 DTFs matched; the build had no warnings

Runs 1 and 6 were run by David. Runs 2 to 5 were run in Raihan's Claude workspace, a Linux copy of the repository (g++ 13.3.0), with the same script (see `ai_usage/phase3/`). Each run's evidence is listed under "Evidence" below.

Test-infrastructure gap (not Front End failures)

All 131 tests in run 1 reported `TERMINAL_EXPECTED_MISSING`. This means the exact terminal expected files had not been written. It says nothing about the Front End's behaviour, so these are not logged as 131 failures.

Observed failures

Test	What it tests	Failure observed	Root cause category	Actual cause	Fix	Status
`sell_15`	A game sold in one session can be bought in a later session	Run 1: `DTF_DIFF session 2`. Expected session 2 DTF had the `04` record below; actual had only the `00` record. Terminal showed `Error: game is not available.`	Test design	One Front End run cannot show the later-session effect. Each login re-reads the master files, the Front End never writes them, and the Back End that would add the game is not part of the run. Design Table 8 notes this	Split into `sell_15a` and `sell_15b`, one run per session. A `sell_15b` fixture lists the game, standing in for the Back End. Expected DTF records unchanged	Test corrected. Both DTFs match from run 2; both tests pass from run 4 and in run 6
`refund_15`	A buyer can buy a refunded game again in a later session	Run 1: `DTF_DIFF session 2`. Expected session 2 DTF had the `04` record below; actual had only the `00` record. Terminal showed `Error: buyer already owns this game.`	Test design	Same limitation. The unchanged collection file still lists Buyer1 as owning Minecraft. Design Table 8 notes this	Split into `refund_15a` and `refund_15b`. A `refund_15b` fixture removes Buyer1's Minecraft, standing in for the Back End. Expected DTF records unchanged. Relies on open question R4	Test corrected. Both DTFs match from run 2; both tests pass from run 4 and in run 6

Test	What it tests	Failure observed	Root cause category	Actual cause	Fix	Status
F1: every test that logs in (100 in run 2, all 150 in run 3)	Login accepted	Runs 2 and 3: `TERMINAL_DIFF`. Printed `Login successful.`; expected `Login successful. Welcome, <username> (<type>).`	Code: wrong message	`LoginTransaction` printed its own placeholder text instead of the catalogue line	Prints the catalogue line with the account's type code	Fixed in run 4; confirmed in run 6
F2: `login_05`, `login_09` (runs 2 and 3)	Unknown and blank username	Runs 2 and 3: printed `Error: account does not exist.`; expected `Error: "<username>" is not a registered user.` (blank shows `""`)	Code: wrong message	Same as F1	Catalogue error line, quoting the typed name	Fixed in run 4; confirmed in run 6
F3: run 2: `invalid_input_06`, `list_accounts_06`, `transaction_output_01`, `07`, `09`, `17`, `18`; run 3 also `create_01–04`, `create_06–11` (17 tests)	create	Prompt `Enter account type:`; messages `Account created successfully.`, `Error: invalid username.`, `Error: username already exists.`, `Error: invalid account type.`	Code: wrong message	`CreateTransaction` used its own prompt and messages	Catalogue prompt (lists the four types), success line with the type code and 0.00 credit, and the three catalogue errors in the catalogue's order	Fixed in run 4; confirmed in run 6
F4: run 2: `list_accounts_07`, `transaction_output_02`, `12`, `17`; run 3 also `delete_01`, `delete_03–07` (10 tests)	delete	Prompt `Enter username:`; messages `Account deleted successfully.`, `Error: account does not exist.`, `Error: cannot delete the currently logged-in account.`	Code: wrong message	`DeleteTransaction` used its own prompt and messages	`Enter username to delete:`, `Account deleted: <username>.` and the two catalogue errors	Fixed in run 4; confirmed in run 6
F5: run 2: `invalid_input_03`, `transaction_output_03`, `14`; run 3 also `sell_01–03`, `sell_05–15a` (17 tests)	sell	`Game listed successfully.`, `Error: invalid game name.`, `Error: game name already exists.`; a bad price gave `Error: invalid price.` or `Error: price must be between 0.00 and 999.99.` (two messages where the catalogue has one)	Code: wrong message	`SellTransaction` used its own messages	`Game listed: <game> for <price>.`, the catalogue name and duplicate errors, and one price error for both an invalid and an out-of-range price	Fixed in run 4; confirmed in run 6

Test	What it tests	Failure observed	Root cause category	Actual cause	Fix	Status
F6: run 2: `buy_01–03`, `buy_05–13`, `sell_10`, `transaction_output_04`, `08`, `15`; run 3 also `delete_05`, `refund_15b`, `sell_15b` (19 tests)	buy	Prompt `Enter seller username:`; `Game purchased successfully.`; three different errors (`game is not available`, `seller does not match listing`, `seller account does not exist`) where the catalogue has one; `cannot buy your own game`; `buyer already owns this game`; `insufficient credit`	Code: wrong message	`BuyTransaction` used its own prompt and messages, and split the first check into three	`Enter seller's username:` (as in design Figure 6), `Purchase successful: <game> from <seller> for <price>.`, one `"<game>" is not for sale by "<seller>"` error, and the other catalogue errors	Fixed in run 4; confirmed in run 6
F7: run 2: `invalid_input_05`, `transaction_output_05`, `07`, `16`, `18`; run 3 also `refund_01`, `refund_03–15a` (19 tests)	refund	Prompts `Enter buyer username:` and `Enter seller username:`; `Refund completed successfully.`; errors `buyer account does not exist`, `seller account does not exist`, `invalid refund amount`, `refund amount must be greater than zero`, `seller has insufficient credit`	Code: wrong message	`RefundTransaction` used its own prompts and messages	Catalogue prompts, `Refund successful: <amount> moved from <seller> to <buyer>.` and the five catalogue errors	Fixed in run 4; confirmed in run 6
F8: runs 2 and 3: `addcredit_01–11`, `invalid_input_04`, `08`, `list_accounts_08`, `transaction_output_06`, `07`, `08`, `10`, `17`, `18` (20 tests)	addcredit	Prompts `Enter amount:` and (admin) `Enter username:`; `Credit added successfully.`; errors `invalid credit amount`, `credit amount must be greater than zero`, `account does not exist`, `session credit limit exceeded`, `maximum account balance exceeded`	Code: wrong message	`AddCreditTransaction` used its own prompts and messages	`Enter amount of credit to add:`, `Enter username to credit:`, `Credit added: <amount> to <username>.` and the five catalogue errors	Fixed in run 4; confirmed in run 6

Test	What it tests	Failure observed	Root cause category	Actual cause	Fix	Status
F9: runs 2 and 3: `list_games_01–06`, `list_games_09–12`	list_games	No header; each game printed as `Minecraft | Seller1 | $50.00`; empty list printed `No games available.`	Code: wrong format	`ListGamesTransaction` used its own line format	Header `Games available for purchase:`, then name padded to 26, seller padded to 15, price; empty list prints `No games are available for purchase.` One helper formats the line, also used by SEARCH	Fixed in run 4; confirmed in run 6
F10: runs 2 and 3: `list_accounts_01`, `06–08`	list_accounts	No header; each account printed as `AdminUser | AA | $1000.00`	Code: wrong format	`ListAccountsTransaction` used its own line format	Header `Active accounts:`, then username padded to 15, type code, credit	Fixed in run 4; confirmed in run 6
F11: runs 2 and 3: `search_01–09`	SEARCH (client change)	Every `search` line printed `Error: "search" is not a valid transaction code.`, and the next line (the search text) was then read as a transaction code and refused the same way. In `search_08` (before login) the login gate message was expected instead	Missing feature	SEARCH was not implemented	New `SearchTransaction`, registered as the eleventh transaction. Blank text is refused; matching games in `list_games` format, then matching usernames; a non-admin is not shown an admin account with no game for sale. No DTF record	Fixed in run 4; confirmed in run 6
F12: `addcredit_12`, `addcredit_13` (new in run 5)	An amount too large to store; an amount with no digit before the point	Against the run 3 build: `addcredit_12` printed `Error: credit amount must be greater than zero.` for `99999999999999999`, because the amount overflowed to a negative number. `addcredit_13` accepted `.5`, printed `Credit added successfully.` and wrote `06 FullUser 000000.50` (`DTF_DIFF session 1`)	Code: logic error	`Money::parse` multiplied without a size limit, and read a missing whole-number part as 0	`Money::parse` follows catalogue rule 5 (digits first, then at most two decimals) and sets an amount too large to store to a value above every limit, so the limit checks reject it	Predicted from reading `Money.cpp` (earlier draft), fixed in run 4, tests added in run 5. Both fail on the old build and pass on the fixed build; confirmed in run 6

F2 to F8 also show the F1 login line, so each of those tests failed on more than one line in runs 2 and 3.

Observed DTF mismatches (run 1)

Session 1 matched for both tests. In session 2:

`sell_15`, from `sell_15_session2_dtf_diff.txt`:

```
1d0
< 04 NextSessionGame            FullUser        BuyUser         025.00
```

`refund_15`, from `refund_15_session2_dtf_diff.txt`:

```
1d0
< 04 Minecraft                  Seller1         Buyer1          050.00
```

Observed DTF mismatch (run 5, against the run 3 build)

`addcredit_13`, from `addcredit_13_session1_dtf_diff.txt`:

```
0a1
> 06 FullUser           000000.50
```

Test corrections (not observed failures)

Test	Problem with the test	Correction	Why it is not in the failure log
`buy_13`	Meant to check that a seller cannot buy their own game, but the shared data made Seller1 sell-standard. The account-type permission check refused `buy` first, so the own-game rule was never reached. Design Table 8 notes this	Fixture `phase1/test_fixtures/buy_13/current_users.txt` makes Seller1 full-standard; all other data is shared. Input and expected DTF unchanged. Expected terminal error: `Error: you cannot buy your own game.`	Run 1 passed its DTF check and had no terminal comparison, so no failure was ever observed
The 49 create, delete, refund and sell terminal expected files, and `sell_10`	The 49 files used the Front End's own wording (for example `Login successful.` and `Account created successfully.`) instead of the catalogue, so all 49 tests passed in run 2 against code that did not follow the catalogue. `sell_10` was a sample written before the catalogue was adopted	Rewritten from the catalogue, each test's input and its data files. All 150 terminal files were then checked against a model of the catalogue rules that does not run the Front End; all agreed	No Front End failure was hidden: the 49 tests all failed in run 3 once corrected, and those failures are F1, F3, F4, F5, F6 and F7. The 49 passes in run 2 are not counted

Predicted, not yet observed

None remain. The three predictions in the earlier draft were all observed: the login line and the buy prompt and success line in run 2 (F1, F6), and the overflow in run 5 (F12).

Source of terminal expected wording

Every `*_terminal_expected.txt` file is written from the test's input, its prose expected result, the data files it runs with (shared data or its fixture), and `phase3/message_catalogue.md`. The catalogue was adopted by the group on 3 November 2026 and is the reference for all terminal wording and the order of checks. No expected file was copied from Front End output. Run 2 found 50 files that did not follow this rule; they were rewritten (see "Test corrections").

Wording taken directly from the Phase 2 design (the catalogue repeats it unchanged)

Text	Design source
`Error: "<code>" is not a valid transaction code.`	Table 5
`Error: please log in before using "<code>".`	Table 5
`Error: already logged in. Log out before logging in again.`	Table 5
`Error: your account type cannot use "<code>".`	Table 5
`Enter game name:` (buy)	Figure 6, step 7
`Enter seller's username:`	Figure 6, step 9
`Logout successful.`	Figure 6, step 32

Wording adopted by the group in the catalogue

The design defines the behaviour (the prompts, their order and the checks) but not these words. Examples:

Text	Used for
`Login successful. Welcome, <username> (<type>).`	Login accepted
`Error: "<username>" is not a registered user.`	Unknown account
`Purchase successful: <game> from <seller> for <price>.`	Buy accepted. Figure 6 shows only the start, `Purchase successful ...`; the catalogue completes it
`Enter amount of credit to add:`, `Credit added: <amount> to <username>.`	Addcredit
`Games available for purchase:`, then fixed-width lines	`list_games`

See the catalogue for the full list. An empty value is shown as `""`, for example `Error: "" is not a registered user.` (group decision, 3 November 2026).

Resolved by the message catalogue

Former question	Resolution
Seller prompt: design `Enter seller's username:` or source `Enter seller username:`	`Enter seller's username:`, as in the design
Buy success text, which Figure 6 shows only in part	`Purchase successful: <game> from <seller> for <price>.`

Former question	Resolution
Approval of the terminal wording	The catalogue was adopted by the group on 3 November 2026
`list_games` line format	Header line, then name padded to 26, seller padded to 15, then the price

Evidence

Run	Folder	Contents
1	`phase3/run1_evidence/`	Summary and the two session-2 DTF diffs
2	`phase3/run2_evidence/`	Console output and summary. Per-test outputs were not kept
3	`phase3/run3_evidence/`	Console output, summary, and all 150 terminal diffs in one file
4	`phase3/run4_evidence/`	Console output and summary. Per-test outputs were not kept
5	`phase3/run5_evidence/`	Console output and summary for the fixed build and for the old-build check, and the diffs of the two new tests
6	`phase3/run6_evidence/`	Run information, console output, summary, and every actual terminal and DTF output

Open requirement questions

ID	Question	Who decides	Affects	How we will address it
R4	A refund names no game. Which game leaves the buyer's collection, and is that the Back End's job?	Client or TA	`refund_14`, `refund_15b` fixture	Until it is answered, `refund_14` checks only the credit transfer and the `05` record, and the `refund_15b` fixture stands in for the Back End. The Front End does not change either way. If a game must leave the collection, the Back End does it in Phase 4 and we add a Back End test for it
R5	SEARCH: does "items" mean only games for sale, or also games in users' collections?	Client (asked on Slack, 3 November)	`SearchTransaction`, `search_01–09`	We assume games for sale. If collections count too, `SearchTransaction` also searches the collection and we add tests; the nine tests stay valid
R6	SEARCH: is matching case-sensitive?	Client (asked on Slack, 3 November)	`search_09`	We assume case-sensitive. If not, one comparison in `SearchTransaction` changes, and so does the expected result of `search_09`
R7	SEARCH: does it write a DTF record?	Client (asked on Slack, 3 November)	DTF expected files of `search_01–09`	We assume no record: the DTF format has no code for it, as for `list_games`. If a record is wanted, we need its format from the client before the Back End phase